

Explicación del código de Tránsito de Datos

En esta actividad se desarrolló un script en Python para procesar datos de tráfico vehicular con el objetivo de generar un conjunto de datos multimodal que pueda utilizarse en modelos de aprendizaje automático. Este trabajo forma parte del proyecto basado en datos de sensores de tráfico similares a los utilizados en los conjuntos METR-LA y PEMS-BAY, los cuales contienen información temporal sobre la velocidad registrada por sensores instalados en carreteras.

Origen de los datos

Para esta actividad se utilizó un archivo llamado `traffic.csv`, el cual contiene datos simulados de sensores de tráfico. Este archivo incluye las siguientes columnas principales:

- `sensor_id`: identificador del sensor
- `datetime`: fecha y hora de la medición
- `speed`: velocidad registrada por el sensor

Se utilizó este archivo porque el proyecto requiere trabajar con datos estructurados de tráfico en formato tabular, lo cual permite aplicar procesamiento temporal, detección de eventos y generación de información adicional.

El archivo se guarda en formato CSV porque es un formato estándar compatible con Python y la librería `pandas`, la cual se utiliza para manipular datos.

Carga de datos en Python

El primer paso del código consiste en cargar el archivo CSV utilizando la librería `pandas`. Esto permite convertir el archivo en un `DataFrame`, que es una estructura de datos que facilita el análisis y la manipulación de la información.

Después de cargar los datos, se convierte la columna `datetime` a formato de fecha para poder ordenar correctamente los registros en el tiempo. Esto es necesario porque el proyecto trabaja con series temporales y el orden cronológico es importante.

Ordenamiento temporal y por sensor

Los datos se ordenan por `sensor_id` y por `datetime` para asegurar que cada sensor tenga sus mediciones en el orden correcto. Esto es importante porque posteriormente se calcula la diferencia de velocidad entre mediciones consecutivas, lo cual solo tiene sentido si los datos están ordenados.

Generación de descripción textual del tráfico

Se creó una función que recibe la velocidad y genera una descripción en texto del estado del tráfico. Esta función permite transformar datos numéricos en información textual, lo cual es necesario para crear un conjunto de datos multimodal.

Las reglas utilizadas fueron:

- velocidad menor a 20 → congestión severa
- velocidad entre 20 y 60 → tráfico moderado
- velocidad mayor a 60 → flujo libre

Esta descripción se guarda en una nueva columna llamada `incident_description`.

Detección de cambios bruscos de velocidad

Después se calcula la diferencia de velocidad entre mediciones consecutivas usando la función `diff()` agrupada por sensor. Esto permite detectar cambios bruscos que podrían indicar un accidente o un evento anormal.

Si la diferencia de velocidad es mayor a cierto umbral, se genera un mensaje indicando posible accidente. Este mensaje reemplaza la descripción normal del tráfico para reflejar el evento detectado.

Este paso es importante porque el proyecto requiere simular información de incidentes, no solo velocidades.

Limpieza de datos

Se eliminan valores vacíos para evitar errores en el procesamiento. Esto garantiza que el DataFrame final tenga únicamente datos válidos.

También se reinicia el índice para mantener el orden correcto después de eliminar filas.

Creación del DataFrame maestro

Se crea una copia del DataFrame limpio para usarlo como conjunto principal de datos. Este DataFrame contiene:

- información numérica
- información temporal
- información textual
- detección de eventos

Este conjunto se considera multimodal porque combina diferentes tipos de datos.

División en train, validation y test

El conjunto de datos se divide en tres partes:

- 70% entrenamiento
- 15% validación
- 15% prueba

La división se hace respetando el orden temporal, lo cual es importante en datos de tráfico, porque no se deben mezclar datos del futuro con datos del pasado.

El conjunto de entrenamiento se usa para entrenar modelos, el de validación para ajustar parámetros, y el de prueba para evaluar resultados.

Exportación de archivos

Finalmente, cada conjunto se guarda en archivos CSV:

- train.csv
- val.csv
- test.csv

Esto permite utilizar los datos en otras etapas del proyecto, como modelos de predicción o análisis de grafos.

Conclusión

El código desarrollado permite transformar datos de tráfico en un conjunto multimodal estructurado, listo para ser utilizado en modelos de aprendizaje automático. Se generaron descripciones textuales, detección de incidentes, orden temporal y división en conjuntos de entrenamiento, validación y prueba.

Este proceso es necesario para preparar los datos antes de aplicar modelos más avanzados en las siguientes etapas del proyecto.